

Lecture 8

Finite Difference Methods for Convection-Diffusion

In this lecture, we introduce the finite difference method for the solution of PDE's. We will limit our discussion of PDE's to convection-diffusion. Recall from Equation 7.6 that the convection equation is,

$$\frac{\partial U}{\partial t} + u \frac{\partial U}{\partial x} + v \frac{\partial U}{\partial y} = 0,$$

where U is the scalar quantity which is convected. Adding diffusion to this equation gives the convection-diffusion equation,

$$\frac{\partial U}{\partial t} + u \frac{\partial U}{\partial x} + v \frac{\partial U}{\partial y} = \mu \left(\frac{\partial^2 U}{\partial x^2} + \frac{\partial^2 U}{\partial y^2} \right), \quad (8.1)$$

where μ is the diffusion coefficient.

8.1 Finite Difference Approximations

The finite difference approximation of a PDE is constructed using a grid over the domain of interest. The finite difference approximations are easiest to derive using a structured, rectangular grid as shown in Figure 8.1. For a finite difference approximation of a PDE, the solution is sought at the nodes of the mesh. We use the notation that $U_{i,j}$ is the value of U at the (i, j) node.

A common finite difference approximation of $\partial U / \partial x$ at node (i, j) is,

$$\left. \frac{\partial U}{\partial x} \right|_{i,j} \approx \delta_{2x} U_{i,j} \equiv \frac{1}{2\Delta x} (U_{i+1,j} - U_{i-1,j}). \quad (8.2)$$

The finite difference operator δ_{2x} is called a central difference operator. Finite difference approximations can also be one-sided. For example, a backward difference approximation is,

$$\left. \frac{\partial U}{\partial x} \right|_{i,j} \approx \delta_x^- U_{i,j} \equiv \frac{1}{\Delta x} (U_{i,j} - U_{i-1,j}), \quad (8.3)$$

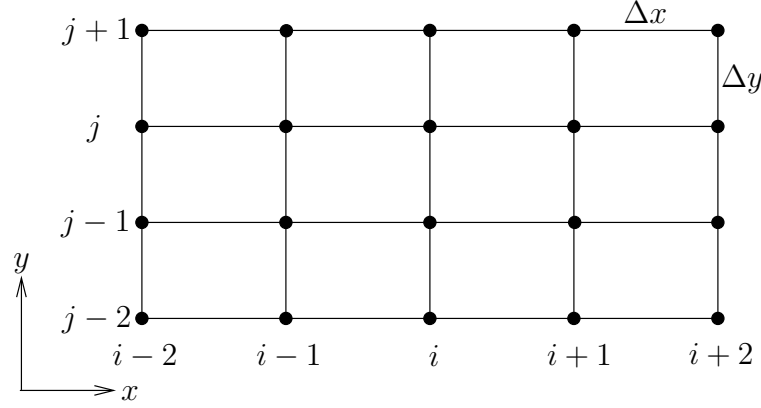


Figure 8.1: Two-dimensional structured mesh for finite difference approximations.

and a forward difference approximation is,

$$\left. \frac{\partial U}{\partial x} \right|_{i,j} \approx \delta_x^+ U_{i,j} \equiv \frac{1}{\Delta x} (U_{i+1,j} - U_{i,j}), \quad (8.4)$$

A very common finite difference approximation for a second derivative is,

$$\left. \frac{\partial^2 U}{\partial x^2} \right|_{i,j} \approx \delta_x^2 U_{i,j} \equiv \frac{1}{\Delta x^2} (U_{i+1,j} - 2U_{i,j} + U_{i-1,j}). \quad (8.5)$$

As in the first derivative case, since this derivative approximation uses both $i+1$ and $i-1$ values, it is known as a central difference approximation of the second derivative. This approximation can be motivated by approximating the second derivatives as a difference of first derivatives,

$$\begin{aligned} \frac{\partial^2 U}{\partial x^2}(x) &= \lim_{\Delta x \rightarrow 0} \frac{\frac{\partial U}{\partial x}(x + \Delta x/2) - \frac{\partial U}{\partial x}(x - \Delta x/2)}{\Delta x} \\ &= \lim_{\Delta x \rightarrow 0} \frac{1}{\Delta x} \left[\frac{U(x + \Delta x) - U(x)}{\Delta x} - \frac{U(x) - U(x - \Delta x)}{\Delta x} \right] \\ &= \lim_{\Delta x \rightarrow 0} \frac{1}{\Delta x^2} [U(x + \Delta x) - 2U(x) + U(x - \Delta x)]. \end{aligned}$$

To approximate the convection-diffusion equation we can combine various finite difference derivative approximations. For example, consider the one-dimensional convection-diffusion equation,

$$\frac{\partial U}{\partial t} + u \frac{\partial U}{\partial x} = \mu \frac{\partial^2 U}{\partial x^2}. \quad (8.6)$$

Approximating this using central differences for all derivatives, the convection-diffusion equation at node i can be approximated as,

$$\frac{dU_i}{dt} + u_i \delta_{2x} U_i = \mu \delta_x^2 U_i. \quad (8.7)$$

This is an ordinary differential equation for U_i which is coupled to the nodal values at $U_{i\pm 1}$. To make this a fully discrete approximation, we need to discretize in time. To do this, we could apply any of the ODE integration methods that we discussed previously. For example, the simple forward Euler integration method would give,

$$\frac{U_i^{n+1} - U_i^n}{\Delta t} + u_i^n \delta_{2x} U_i^n = \mu \delta_x^2 U_i^n. \quad (8.8)$$

This method is widely known as a Forward Time-Central Space (FTCS) approximation.

In finite differences, we often make use of the spatial stencil of a discretization. The spatial stencil is simply the subset of nodes which are used to discretize the problem in space. The central difference approximation in Equation 8.7 gives a three-point spatial stencil since dU/dt at node i depends on three nodal values of U (including itself), specifically U_{i-1} , U_i , and U_{i+1} . To illustrate this, note that Equation (8.7) can be written as,

$$\frac{dU_i}{dt} = \alpha_{i-1} U_{i-1} + \beta_i U_i + \gamma_{i+1} U_{i+1}$$

where

$$\begin{aligned} \alpha_{i-1} &= \frac{u_i}{2\Delta x} + \frac{\mu}{\Delta x^2}, \\ \beta_i &= -2\frac{\mu}{\Delta x^2}, \\ \gamma_{i+1} &= -\frac{u_i}{2\Delta x} + \frac{\mu}{\Delta x^2}, \end{aligned}$$

We can also place all of the nodal states into a vector,

$$U = (U_1, U_2, \dots, U_{i-1}, U_i, U_{i+1}, \dots, U_{N_x-1}, U_{N_x})^T,$$

where N_x is the total number of points in the x -direction. Then, the finite difference approximation of convection-diffusion can be written in the following form,

$$\frac{dU}{dt} = AU + b, \quad (8.9)$$

where b will contain boundary condition related data (boundary conditions are discussed in Section 8.2) and the matrix A is,

$$A = \begin{pmatrix} A_{1,1} & A_{1,2} & A_{1,3} & \dots & A_{1,N_x} \\ A_{2,1} & A_{2,2} & A_{2,3} & \dots & A_{2,N_x} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A_{N_x,1} & A_{N_x,2} & A_{N_x,3} & \dots & A_{N_x,N_x} \end{pmatrix}$$

Note that row i of this matrix contains the coefficients of the nodal values for the ODE governing node i . Except for rows requiring boundary condition data, the values of $A_{i,j}$ are related to the coefficients γ_{i+1} , β_i and α_{i-1} , specifically,

$$A_{i,i-1} = \alpha_{i-1}, \quad A_{i,i} = \beta_i, \quad A_{i,i+1} = \gamma_{i+1},$$

and all other entries in row i are zero.

8.2 Boundary Conditions

In this section, we discuss the implementation of finite difference methods at boundaries. This discussion is not meant to be comprehensive, as the issues are many and often subtle. In particular, we only focus on Dirichlet boundary conditions.

A Dirichlet boundary condition is one in which the state is specified at the boundary. For example, in a heat transfer problem, the temperature might be known at the boundary. Dirichlet boundary conditions can be implemented in a relatively straightforward manner. For example, suppose that we are solving a one-dimensional convection-diffusion problem and we want the value of U at $i = 1$, to be U_{inlet} ,

$$U_1 = U_{inlet}.$$

To implement this, we fix $U_1 = U_{inlet}$ and apply the finite difference discretization only over the interior of the computational domain accounting for the known value of U_1 at any place where the interior discretization depends on it. For example, at the first interior node (i.e. $i = 2$), the central difference discretization of 1-D convection-diffusion gives,

$$\frac{dU_2}{dt} + u_2 \frac{U_3 - U_1}{2\Delta x} = \mu \frac{U_3 - 2U_2 + U_1}{\Delta x^2}.$$

Accounting for the known value of U_1 , this becomes,

$$\frac{dU_2}{dt} + u_2 \frac{U_3 - U_{inlet}}{2\Delta x} = \mu \frac{U_3 - 2U_2 + U_{inlet}}{\Delta x^2}. \quad (8.10)$$

In terms of the vector notation, when a Dirichlet boundary condition is applied we usually remove that state from the vector U . So, in the situation where U_1 is known, the state vector is defined as,

$$U = (U_2, U_3, \dots, U_{i-1}, U_i, U_{i+1}, \dots, U_{N_x-1}, U_{N_x})^T,$$

The b vector then will contain the contributions from the known boundary values. For example, by re-arranging Equation (8.10), the first row of b contains,

$$b_1 = u_2 \frac{U_{inlet}}{2\Delta x} + \mu \frac{U_{inlet}}{\Delta x^2}.$$

Since U_{inlet} does not enter any of the other node's stencils, the remaining rows of b will be zero (unless they are altered by the other boundary). Note, the first row of A is,

$$A_{1,1} = -2\frac{\mu}{\Delta x^2}, \quad A_{1,2} = -\frac{u_2}{2\Delta x} + \frac{\mu}{\Delta x^2},$$

and $A_{1,j} = 0$ for $j > 2$.

Example 8.1 (Finite Difference Method applied to 1-D Convection) *In this example, we solve 1-D convection,*

$$\frac{\partial U}{\partial t} + u \frac{\partial U}{\partial x} = 0,$$

using a central difference spatial approximation with a forward Euler time integration,

$$\frac{U_i^{n+1} - U_i^n}{\Delta t} + u_i^n \delta_{2x} U_i^n = 0.$$

Note: this approximation is the Forward Time-Central Space method from Equation (8.8) with the diffusion terms removed.

We will solve a problem that is nearly the same as that in Example 7.4. Specifically, the initial condition is

$$U_0(x) = e^{-x^2}.$$

We let the velocity, $u = 1$. Instead of solving a problem with periodicity, we enforce the inflow boundary (which is at $x = x_L$ since $u > 0$),

$$U(t, x_L) = e^{-x_L^2}.$$

At the outflow, since $u > 0$, no boundary condition is needed at $x = x_R$ (i.e. the scalar just convects out the domain at this location). But, the central difference discretization cannot be applied at the outlet node (i.e. at $i = N_x$) because it would require the $N_x + 1$ nodal value (which does not exist). To handle this problem, we switch the discretization at $i = N_x$ to a one-sided backwards difference formula.

A Matlab script that implements this algorithm is:

```
% This Matlab script solves the one-dimensional convection
% equation using a finite difference algorithm. The
% discretization uses central differences in space and forward
% Euler in time. An inflow bc is set and at the outflow a
% one-sided (backwards) difference is used. The initial condition
% is a Gaussian distribution.
%

clear all;

% Set-up grid
xL = -4;
xR = 4;
Nx = 51; % number of control volumes
x = linspace(xL,xR,Nx);

% Calculate cell size in control volumes (assumed equal)
dx = x(2) - x(1);

% Set velocity
u = 1;

% Set final time
tfinal = 100;
```

```

% Set timestep
CFL = 0.1;
dt = CFL*dx/abs(u);

% Set initial condition
U = exp(-x.^2);
t = 0;

% Set bc state at left (assumes u>0)
UL = exp(-xL^2);

% Loop until t > tfinal
while (t < tfinal),

    % Copy old state vector
    U0 = U;

    % Inflow boundary
    U(1) = UL;

    % Interior nodes
    for i = 2:Nx-1,
        U(i) = U0(i) - dt*u*(U0(i+1)-U0(i-1))/(2*dx);
    end

    % Outflow boundary uses backward difference
    i = Nx;
    U(i) = U0(i) - dt*u*(U0(i)-U0(i-1))/(dx);

    % Increment time
    t = t + dt;

    % Plot current solution
    plot(x,U,'*');
    xlabel('x'); ylabel('U');
    title(sprintf('time = %f\n',t));
    axis([xL, xR, -0.5, 1.5]);
    grid on;
    drawnow;

end

```

When this method is run, the initial Gaussian disturbance convects out the domain, however small oscillations are observed which start at the outlet and move upstream (i.e. to

the left). Eventually, these oscillations grow until the entire solution is contaminated. In Lecture 9 we will show that the FTCS algorithm is unstable for any Δt for pure convection. Thus, what we are observing is an instability that can be predicted through some analysis.

8.3 Truncation Error Analysis

In the discussion of ODE integration, we used the ideas of consistency and stability to prove convergence through the Dahlquist Equivalence Theorem. Similar concepts also exist for PDE discretizations, however, we cannot cover these here. We will briefly look at the local truncation error for finite difference approximations of derivatives, and as well the governing partial differential equations.

8.3.1 Truncation Error for a Derivative Approximation

Suppose we use a backwards difference, $\delta_x^- U_i$ to approximate the first derivative, U_x at point i . The local truncation error for this derivative approximation can be calculated using Taylor series as we have done in the past:

$$\begin{aligned}\tau &\equiv \delta_x^- U_i - U_{xi}, \\ &= \frac{1}{\Delta x} (U_i - U_{i-1}) - U_{xi}, \\ &= \frac{1}{\Delta x} \left[U_i - \left(U_i - \Delta x U_{xi} + \frac{1}{2} \Delta x^2 U_{xxi} + O(\Delta x^3) \right) \right] - U_{xi}, \\ &= -\frac{1}{2} \Delta x U_{xxi} + O(\Delta x^2).\end{aligned}$$

Thus, the analysis shows that the backwards difference is a first-order accurate discretization of the derivative at node i .

8.3.2 Truncation Error for a PDE

Similar to the ODE case, the truncation error is defined as the remainder after an exact solution to the governing equation is substituted into the finite difference approximation. For example, suppose we are using the FTCS algorithm in Equation (8.8) to approximate the one-dimensional convection-diffusion equation. Then the local truncation error for the PDE approximation is defined as,

$$\tau \equiv \frac{U_i^{n+1} - U_i^n}{\Delta t} + u_i^n \delta_{2x} U_i^n - \mu \delta_x^2 U_i^n, \quad (8.11)$$

where $U(x, t)$ is an exact solution to Equation (8.6). Note that the truncation error defined here for PDE's is not quite a direct analogy with the standard definition of local truncation error used in ODE integration, specifically in Equation (2.1). In particular, in the ODE case, the truncation error is defined as the difference between the numerical solution and the exact solution after one step of the method (starting from exact values). However, in the PDE case, we have defined the truncation error as the remainder after an exact solution is

substituted into the numerical method when the numerical method is written in a form that approximates the governing PDE.

Except for this difference in the definition, the calculation of the local truncation follows the same procedure as in the ODE case in which Taylor series substitutions are used to expand the error in powers of Δt . Continuing on with our example, we use Taylor series of U about $t = t^n$ and $x = x_i$. However, we can use our previous results from the analysis of the truncation error of spatial derivatives. Specifically, we showed in during in-class discussions that,

$$\begin{aligned}\delta_{2x}U_i &= U_{xi} + \frac{1}{6}\Delta x^2 U_{xxxi} + O(\Delta x^4) \\ \delta_x^2 U_i &= U_{xxi} + \frac{1}{12}\Delta x^2 U_{xxxxi} + O(\Delta x^4)\end{aligned}$$

Using these results,

$$\tau = \frac{U_i^{n+1} - U_i^n}{\Delta t} + u_i^n \left[U_{xi}^n + \frac{1}{6}\Delta x^2 U_{xxxi}^n + O(\Delta x^4) \right] - \mu \left[U_{xxi}^n + \frac{1}{12}\Delta x^2 U_{xxxxi}^n + O(\Delta x^4) \right]$$

Then, performing a similar Taylor series of the time derivative approximation gives,

$$\frac{U^{n+1} - U^n}{\Delta t} = U_t^n + \frac{1}{2}\Delta t U_{tt}^n + O(\Delta t^2).$$

Substituting this into τ and collecting terms in powers of Δt and Δx gives,

$$\begin{aligned}\tau &= U_{ti}^n + u_i^n U_{xi}^n - \mu U_{xxi}^n + \\ &\quad \frac{1}{2}\Delta t U_{tti}^n + O(\Delta t^2) + \\ &\quad \frac{1}{6}\Delta x^2 u_i^n U_{xxxi}^n - \frac{1}{12}\Delta x^2 \mu U_{xxxxi}^n + O(\Delta x^4)\end{aligned}$$

The first line of this equation is actually just the PDE, evaluated at $t = t^n$ and $x = x_i$. Since U is an exact solution to the PDE, this is zero. The second line shows that the time discretization introduces an $O(\Delta t)$. The third line shows that the spatial discretization introduces an $O(\Delta x^2)$ error. Thus, this numerical method is first-order accurate in time and second-order accurate in space.

Lecture 8: In-Class Demos and Discussion Topics

- Finite Volume vs. Finite Difference Discretizations.
- Central difference approximation of U_{xi} .
- Central difference approximation of U_{xxi} .
- Example 8.1: Finite Difference Method applied to 1-D Convection. Matlab code: `convect1d_ftcs.m`.

Lecture 9

Matrix Stability Analysis of Finite Difference Methods

In this lecture, we analyze the stability of PDE discretizations using a matrix approach. This method builds upon our understanding of eigenvalue stability for systems of ODE's.

As we saw in Section 8.1, finite difference (or finite volume) approximations can potentially be written in a semi-discrete form as,

$$\frac{dU}{dt} = AU + b. \quad (9.1)$$

While there are some PDE discretization methods that cannot be written in that form, the majority can be. So, we will take the semi-discrete Equation (9.1) as our starting point. Note: the term semi-discrete is used to signify that the PDE has only been discretized in space.

Let $U(t)$ be the exact solution to the semi-discrete equation. Then, consider perturbation $e(t)$ to the exact solution such that the perturbed solution, $V(t)$, is:

$$V(t) = U(t) + e(t).$$

The questions that we wish to resolve are: (1) can the perturbation $e(t)$ grow in time for the semi-discrete problem, and (2) what the stability limits are on the timestep for a chosen time integration method.

First, we substitute $V(t)$ into Equation (9.1),

$$\begin{aligned} \frac{dV}{dt} &= AV + b \\ \frac{d(U + e)}{dt} &= A(U + e) + b \\ \frac{de}{dt} &= Ae. \end{aligned}$$

Thus, the perturbation must satisfy the homogeneous equation, $e_t = Ae$. Having studied the behavior of linear system of equations in Section 4.2, we know that $e(t)$ will grow unbounded as $t \rightarrow \infty$ if any of the real parts of the eigenvalues of A are positive.

The problem is that determining the eigenvalues of A can be non-trivial. In fact, for a general problem finding the eigenvalues of A can be about as hard as solving the specific problem. So, while the matrix stability method is quite general, it can also require a lot of time to perform. Still, the matrix stability method is an indispensable part of the numerical analysis toolkit.

As we saw in the eigenvalue analysis of ODE integration methods, the integration method must be stable for all eigenvalues of the given problem. One manner that we can determine whether the integrator is stable is by plotting the eigenvalues scaled by the timestep in the complex $\lambda\Delta t$ plane and overlaying the stability region for the desired ODE integrator. In fact, we have already plotted the eigenvalues for one-dimensional diffusion using a central difference discretization in Example 5.1. Then, Δt can be adjusted to attempt to bring all eigenvalues into the stability region for the desired ODE integrator.

Example 9.1 (Matrix Stability of FTCS for 1-d convection) *In Example 8.1, we used a forward time, central space (FTCS) discretization for 1-d convection,*

$$\frac{U_i^{n+1} - U_i^n}{\Delta t} + u_i^n \delta_{2x} U_i^n = 0. \quad (9.2)$$

Since this method is explicit, the matrix A does not need to be constructed directly, rather Equation (9.2) can be used to find the new values of U at each point i . The Matlab script given in Example 8.1 does exactly that. However, if we are interested in calculating the eigenvalues to analyze the eigenvalue stability, then the A matrix is required. The following script does exactly that (i.e. calculates A , determines the eigenvalues of A , and then plots the eigenvalues scaled by Δt overlayed with the forward Euler stability region). The script can set either the inflow/outflow boundary conditions described in Example 8.1, or can set periodic boundary conditions. We will look at the eigenvalues of both cases.

```
% This Matlab script calculates the eigenvalues of
% the one-dimensional convection equation discretized by
% finite differences. The discretization uses central
% differences in space and forward Euler in time.
%
% Periodic bcs are set if periodic_flag == 1.
%
% Otherwise, an inflow (dirichlet) bc is set and at
% the outflow a one-sided (backwards) difference is used.
%

clear all;

periodic_flag = 1;

% Set-up grid
xL = -4;
xR = 4;
```

```

Nx = 21; % number of points
x = linspace(xL,xR,Nx);

% Calculate cell size in control volumes (assumed equal)
dx = x(2) - x(1);

% Set velocity
u = 1;

% Set timestep
CFL = 1;
dt = CFL*dx/abs(u);

% Set bc state at left (assumes u>0)
UL = exp(-xL^2);

% Allocate matrix to hold stiffness matrix (A).
%
A = zeros(Nx-1,Nx-1);

% Construct A except for first and last row
for i = 2:Nx-2,
    A(i,i-1) = u/(2*dx);
    A(i,i+1) = -u/(2*dx);
end

if (periodic_flag == 1), % Periodic bcs

    A(1,2) = -u/(2*dx);
    A(1,Nx-1) = u/(2*dx);
    A(Nx-1,1) = -u/(2*dx);
    A(Nx-1,Nx-2) = u/(2*dx);

else % non-periodic bc's

    % At the first interior node, the i-1 value is known (UL).
    % So, only the i+1 location needs to be set in A.
    A(1,2) = -u/(2*dx);

    % Outflow boundary uses backward difference
    A(Nx-1,Nx-2) = u/dx;
    A(Nx-1,Nx-1) = -u/dx;

end

```

```
% Calculate eigenvalues of A
lambda = eig(A);

% Plot lambda*dt
plot(lambda*dt, '*');
xlabel('Real \lambda\Delta t');
ylabel('Imag \lambda\Delta t');

% Overlay Forward Euler stability region
th = linspace(0,2*pi,101);
hold on;
plot(-1 + sin(th),cos(th));
hold off;
axis('equal');
grid on;
```

Figure 9.1 shows a plot of $\lambda\Delta t$ for a CFL set to one. Recall that for this one-dimensional problem, the CFL number was defined as,

$$CFL = \frac{|u|\Delta t}{\Delta x}.$$

In the inflow/outflow boundary condition case (shown in Figure 9.1) the eigenvalues lay slightly inside the negative real half-plane. As they move away from the origin, they approach the imaginary axis at $\pm i$. The periodic boundary conditions give purely imaginary eigenvalues but these also approach $\pm i$ as they move away from the origin. Note that the periodic boundary conditions actually give a zero eigenvalue so that the matrix A is actually singular (Why is this?). Regardless what we see is that for a $CFL = 1$, some $\lambda\Delta t$ exist which are outside of the forward Euler stability region. We could try to lower the timestep to bring all of the $\lambda\Delta t$ into the stability region, however that will prove to be practically impossible since the extreme eigenvalues approach $\pm\alpha i$ (i.e. they are purely imaginary). Thus, no finite value of Δt exists for which these eigenvalues can be brought inside the circular stability region of the forward Euler method (i.e. the FTCS is unstable for convection).

In-class Discussion 9.1 (Behavior of FTCS eigenvalues with decreased Δx) We will discuss Figure 9.2.

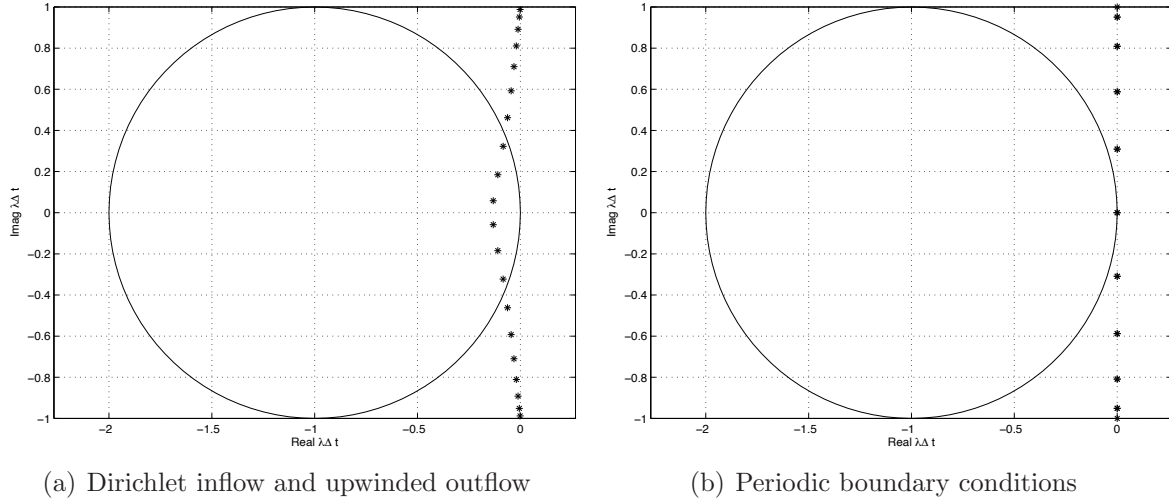


Figure 9.1: $\lambda\Delta t$ distribution for one-dimensional convection example using two different boundary conditions. Note: Δt set such that $\text{CFL} = 1$.

In-class Discussion 9.2 (Behavior of FTCS eigenvalues with diffusion) *We will discuss Figure 9.3.*

Though the eigenvalues of A typically require numerical techniques for the general problem, a special case of practical interest occurs when the matrix is ‘periodic’. That is, the column entries shift a column every row. Thus, the matrix has the form,

$$A = \begin{pmatrix} a_1 & a_2 & a_3 & \dots & a_N \\ a_N & a_1 & a_2 & \dots & a_{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_2 & a_3 & a_4 & \dots & a_1 \end{pmatrix}$$

This type of matrix is known as a circulant matrix. Circulant matrices have eigenvalues given by,

$$\lambda_n = \sum_{j=1}^N a_j e^{i 2\pi(j-1)\frac{n}{N}} \quad \text{for } n = 0, 1, \dots, N-1 \quad (9.3)$$

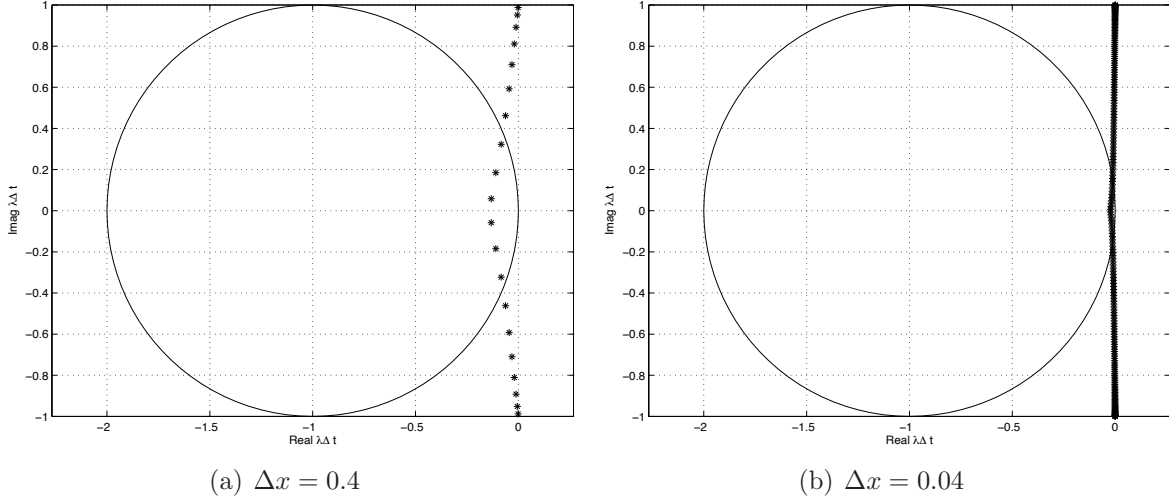


Figure 9.2: Effect of Δx on $\lambda\Delta t$ distribution for one-dimensional convection example using Dirichlet inflow and upwinded outflow conditions. Note: Δt set such that $CFL = 1$.

Example 9.2 As we saw in Example 9.1, when periodic boundary conditions are assumed, the central space discretization of one-dimensional convection gives purely imaginary eigenvalues, and when scaled by a timestep for which the CFL number is one, the eigenvalues stretch along the axis until $\pm i$. Since for a convection problem with constant velocity and periodic boundary conditions gives a circulant matrix, we can use Equation (9.3) to determine the eigenvalues analytically. We begin by finding the coefficients, a_j . For a central space discretization, we find,

$$a_2 = -\frac{u}{2\Delta x}, \quad a_N = \frac{u}{2\Delta x}, \quad \text{and for all other } j, \quad a_j = 0.$$

Then, substituting these a_j into Equation (9.3) gives,

$$\begin{aligned} \lambda_n &= -\frac{u}{2\Delta x} e^{i2\pi \frac{n}{N}} + \frac{u}{2\Delta x} e^{i2\pi(N-1)\frac{n}{N}}, \\ &= -\frac{u}{2\Delta x} e^{i2\pi \frac{n}{N}} + \frac{u}{2\Delta x} e^{i2\pi n} e^{-i2\pi \frac{n}{N}}. \end{aligned}$$

Since $e^{i2\pi n} = 1$ (because n is an integer), then,

$$\begin{aligned} \lambda_n &= -\frac{u}{2\Delta x} e^{i2\pi \frac{n}{N}} + \frac{u}{2\Delta x} e^{-i2\pi \frac{n}{N}}, \\ &= -\frac{u}{2\Delta x} (e^{i2\pi \frac{n}{N}} - e^{-i2\pi \frac{n}{N}}), \\ &= -i \frac{u}{\Delta x} \sin\left(2\pi \frac{n}{N}\right). \end{aligned}$$

Multiplying by the timestep,

$$\lambda_n \Delta t = -i \frac{u \Delta t}{\Delta x} \sin\left(2\pi \frac{n}{N}\right).$$

As observed in Example 9.1, the eigenvalues are purely imaginary and will extend to $\pm i$ when $CFL = |u|\Delta t/\Delta x = 1$.

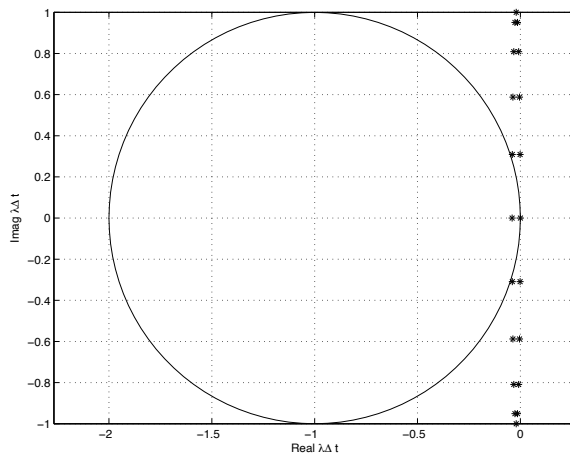
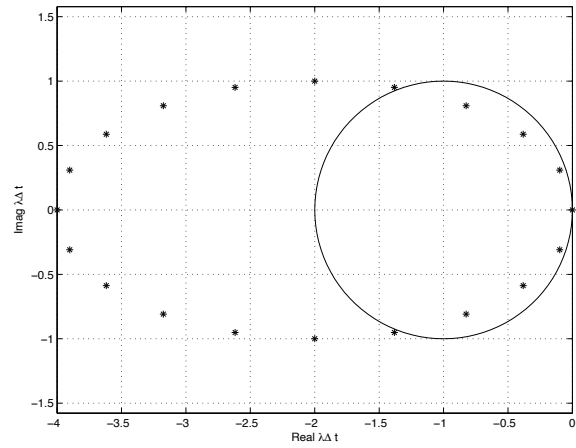
(a) $u\Delta x/\mu = 100$ (b) $u\Delta x/\mu = 1$

Figure 9.3: Effect of viscosity on $\lambda\Delta t$ distribution for one-dimensional convection-diffusion example using periodic boundary conditions. Note: Δt set such that CFL = 1.

Lecture 10

Fourier Analysis of Finite Difference Methods

In this lecture, we determine the stability of PDE discretizations using Fourier analysis. First, we begin with Fourier analysis of PDE's, and then extend these ideas to finite difference methods.

10.1 Fourier Analysis of PDE's

We will develop Fourier analysis in one dimension. The basic ideas extend easily to multiple dimensions. We will consider the convection-diffusion equation,

$$\frac{\partial U}{\partial t} + u \frac{\partial U}{\partial x} = \mu \frac{\partial^2 U}{\partial x^2}.$$

We will assume that the velocity, u , and the viscosity, μ are constant.

The solution is assumed to be periodic over a length L . Thus,

$$U(x + mL, t) = U(x, t)$$

where m is any integer. As we saw in Lecture 9, as the mesh is refined, the eigenvalues of the systems with general boundary conditions tend to approach the eigenvalues of the periodic case. Thus, we expect this periodicity assumption to still lead to insight into more general boundary conditions especially as the mesh is refined.

A Fourier series with periodicity over length L is given by,

$$U(x, t) = \sum_{m=-\infty}^{+\infty} \hat{U}_m(t) e^{ik_m x} \quad \text{where} \quad k_m = \frac{2\pi m}{L}. \quad (10.1)$$

k_m is generally called the wavenumber, though m is the number of waves occurring over the length L . We note that $\hat{U}_m(t)$ is the amplitude of the m -th wavenumber and it is generally complex (since we have used complex exponentials). Substituting the Fourier series into the convection-diffusion equation gives,

$$\frac{\partial}{\partial t} \left[\sum_{m=-\infty}^{+\infty} \hat{U}_m(t) e^{ik_m x} \right] + u \frac{\partial}{\partial x} \left[\sum_{m=-\infty}^{+\infty} \hat{U}_m(t) e^{ik_m x} \right] = \mu \frac{\partial^2}{\partial x^2} \left[\sum_{m=-\infty}^{+\infty} \hat{U}_m(t) e^{ik_m x} \right].$$

$$\sum_{m=-\infty}^{+\infty} \frac{d\hat{U}_m}{dt} e^{ik_m x} + u \sum_{m=-\infty}^{+\infty} ik_m \hat{U}_m e^{ik_m x} = \mu \sum_{m=-\infty}^{+\infty} (ik_m)^2 \hat{U}_m e^{ik_m x}.$$

Noting that $i^2 = -1$ and collecting terms gives,

$$\sum_{m=-\infty}^{+\infty} \left[\frac{d\hat{U}_m}{dt} + (iuk_m + \mu k_m^2) \hat{U}_m \right] e^{ik_m x} = 0. \quad (10.2)$$

The next step is to utilize the orthogonality of the different Fourier modes over the length L , specifically,

$$\int_0^L e^{-ik_n x} e^{ik_m x} dx = \begin{cases} 0 & \text{if } m \neq n \\ L & \text{if } m = n \end{cases} \quad (10.3)$$

By multiplying Equation (10.2) by $e^{-ik_n x}$ and integrating from 0 to L , the orthogonality condition gives,

$$\frac{d\hat{U}_n}{dt} + (iuk_n + \mu k_n^2) \hat{U}_n = 0, \quad \text{for any integer value of } n. \quad (10.4)$$

Thus, the evolution of the amplitude for an individual wavenumber is independent of the other wavenumbers. The solution to Equation (10.4),

$$\hat{U}_n(t) = \hat{U}_n(0) e^{-iuk_n t} e^{-\mu k_n^2 t}.$$

The convection term, which results in the complex time dependent behavior, $e^{-iuk_n t}$, only oscillates and does not change the magnitude of $\hat{U}_n$. The diffusion term causes the magnitude to decrease as long as $\mu > 0$. But, if the diffusion coefficient were negative, then the magnitude would increase unbounded with time. Thus, in the case of the convection-diffusion PDE, as long as $\mu \geq 0$, this solution is stable.

10.2 Semi-Discrete Fourier Analysis

Now we move on to Fourier analysis of a semi-discrete equation. That is, we discretize in space but not time. To be specific and simple, let consider pure convection discretized with central differences. This is exactly the case considered in Example 9.2 so we already know the results. But here, we derive the eigenvalues of the central difference discretization using Fourier analysis. The semi-discrete equation is,

$$\frac{dU_j}{dt} + u \delta_{2x} U_j = 0, \quad \Rightarrow \quad \frac{dU_j}{dt} + \frac{u}{2\Delta x} (U_{j+1} - U_{j-1}) = 0. \quad (10.5)$$

Note that we have switched our indices from the usual i 's to j 's to avoid confusion in this discussion because the Fourier series we are about to introduce will give rise to the imaginary number, i .

For the analysis of PDE's, a Fourier series of infinite dimension was used (i.e. m ranged from $\pm\infty$ in Equation (10.1)). In that case, the infinite number of terms in the Fourier series corresponds to the fact that there are infinitely many values of U over the periodic domain

(because the space from $0 \leq x \leq L$ has an infinite number of points). In the semi-discrete case, only a finite number of values exist over the periodic domain. Specifically, if there are N equally-spaced nodes in the domain, then there will be $N - 1$ unique values (because the values at the beginning and end of the domain must be the same due to periodicity). Summarizing, there can only be $N - 1$ terms in the Fourier series used to describe a finite difference solution on a periodic domain with N points.

The most common set of $N - 1$ modes used to analyze finite difference schemes is,

$$U_j(t) = \sum_{m=-N/2+1}^{N/2-1} \hat{U}_m(t) e^{ik_m j \Delta x} \quad \text{for even } N, \quad (10.6)$$

and,

$$U_j(t) = \sum_{m=-(N-3)/2}^{(N-1)/2} \hat{U}_m(t) e^{ik_m j \Delta x} \quad \text{for odd } N \quad (10.7)$$

Note that for large N , the limits on either Fourier series approach $\pm N/2$.

So what happens to the modes with higher wavenumbers? The answer is that the higher wavenumbers are aliased with the lower wavenumber when viewed only at a finite number of nodes. To demonstrate this, consider the case with $N = 5$ nodes. In this case, the values of m in the Fourier series are from -1 to 2 . The real and imaginary parts of the modes are plotted in Figure 10.1 for $m = 0, 1$, and 2 . Although the mode shape is shown for all values of x , the only values of concern are those at the nodes. We can already see the potential for aliasing by looking at the imaginary part of the $m = 2$ mode. In this case, the nodal values of this mode are all zero. Thus, at the nodes, the imaginary part of $m = 2$ mode is identical to the imaginary part of the $m = 0$ mode (i.e. they are both zero at the nodes). The $m = 2$ mode is not completely aliased with the $m = 0$ mode, however, because the real part of the modes are different. Specifically, the real part of the $m = 0$ mode is constant (equal to one at all nodes), while the real part of the $m = 2$ mode oscillates between ± 1 . This ± 1 oscillation between nodes is often called an odd-even or a sawtooth mode. To demonstrate what happens with higher wave numbers, Figure 10.2 shows the $m = 3$ mode, overlayed with the $m = -1$ mode. As can be seen from this plot, these two modes are indistinguishable from each other at the nodes.

If we proceed along the analogous lines to the Fourier analysis of PDE's in the previous section, we would substitute the Fourier series into Equation (10.5) and utilize a similar orthogonality relationship to arrive at the conclusion that each mode of the Fourier series behaves independently. Thus, from now on, we will simply substitute an individual mode into the discrete equations. That is, we assume that

$$U_j(t) = \hat{U}_m(t) e^{ik_m j \Delta x},$$

for a valid m . Substitution of this individual mode into Equation (10.5) gives,

$$\frac{d\hat{U}_m}{dt} e^{ik_m j \Delta x} + \frac{u}{2\Delta x} [e^{ik_m(j+1)\Delta x} - e^{ik_m(j-1)\Delta x}] \hat{U}_m = 0.$$

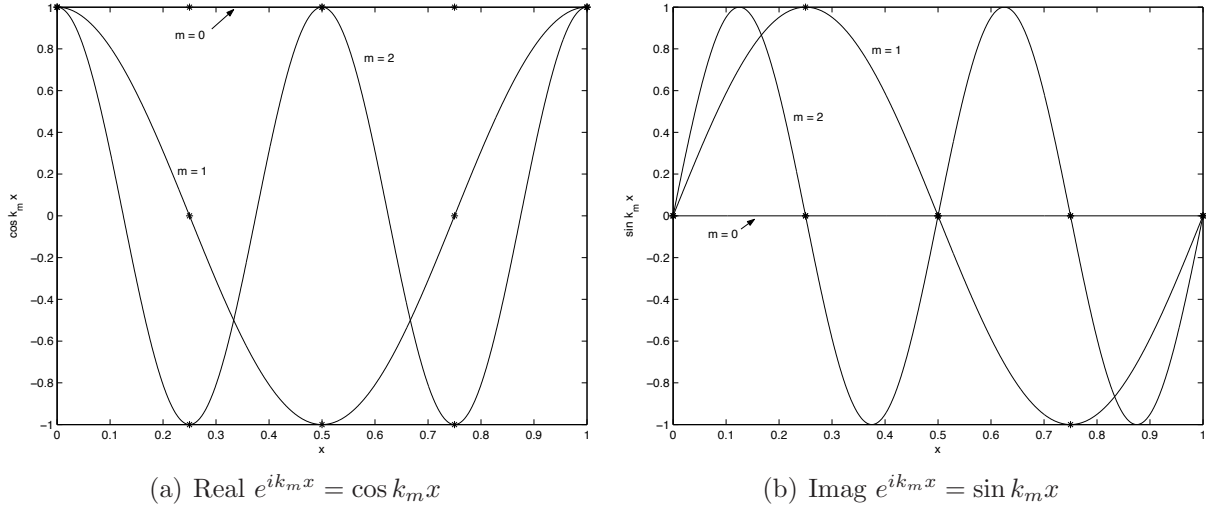


Figure 10.1: Plots of $\cos k_m x$ and $\sin k_m x$ for $L = 1$ including the nodal values for a five node grid ($\Delta x = 0.2$).

Factoring out the term $e^{ik_m j \Delta x}$ gives,

$$\frac{d\hat{U}_m}{dt} + \frac{u}{2\Delta x} [e^{ik_m \Delta x} - e^{-ik_m \Delta x}] \hat{U}_m = 0.$$

$$\frac{d\hat{U}_m}{dt} + i \frac{u}{\Delta x} \sin(k_m \Delta x) \hat{U}_m = 0.$$

Thus, for each mode m we may write,

$$\frac{d\hat{U}_m}{dt} = \lambda_m \hat{U}_m,$$

where

$$\lambda_m = -i \frac{u}{\Delta x} \sin(k_m \Delta x).$$

Comparing these λ_m to the eigenvalues found in Example 9.2, we can show that they are identical.

A parameter which occurs throughout Fourier analysis of spatial discretizations is,

$$\beta_m \equiv k_m \Delta x. \quad (10.8)$$

Since $k_m = 2\pi m/L$, then

$$\beta_m = 2\pi \frac{m \Delta x}{L}.$$

Since β_m varies with m , we can determine the range of β_m values corresponding to the range of m values. In particular, for large N , the limiting values of m approach $\pm N/2$, thus the limiting values of β_m will be,

$$\lim_{\substack{m \rightarrow \pm N/2 \\ N \rightarrow \infty}} \beta_m = \pm 2\pi \frac{N \Delta x}{2L} = \pm \pi \frac{N \Delta x}{L} = \pm \pi \frac{N}{N-1} = \pm \pi.$$

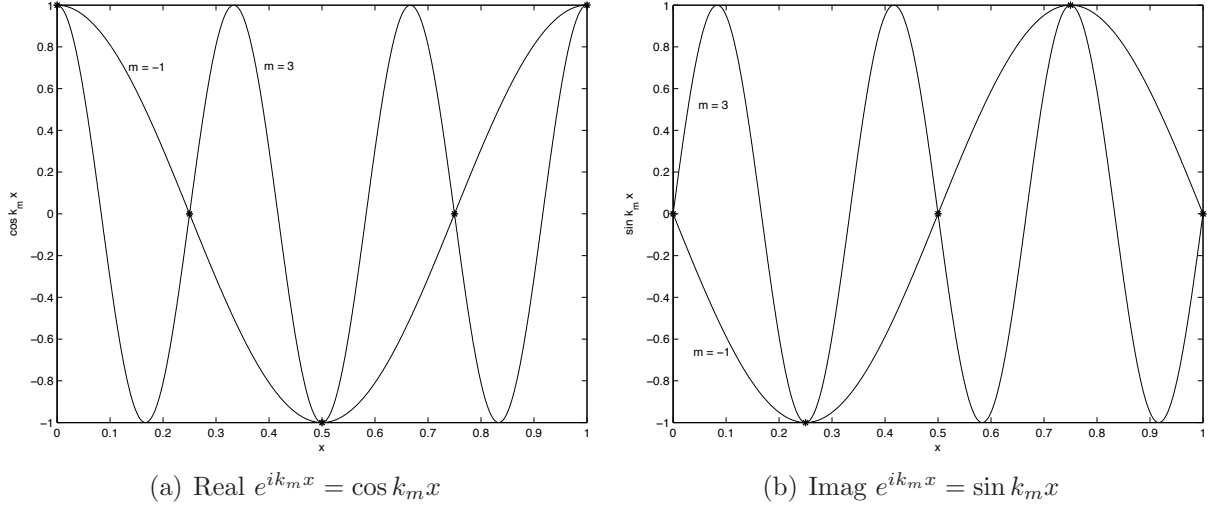


Figure 10.2: Demonstration of aliasing of an $m = 3$ mode to $m = -1$ for a five node grid.

Thus, assuming large N , the eigenvalue calculation for any spatial differencing method reduces to:

1. Substitute $U_j(t) = \hat{U}_m(t)e^{ij\beta_m}$ into the semi-discrete equation.
2. Determine the $\lambda_m(\beta_m)$ such that $\frac{d}{dt}\hat{U}_m = \lambda_m\hat{U}_m$.
3. Determine the limitations on the timestep such that $\lambda_m(\beta_m)\Delta t$ will be inside the stability region for a chosen time integration method for all $-\pi \leq \beta_m \leq \pi$.

Example 10.1 (Analysis of a first-order upwind discretization of convection) *In this example, we perform a Fourier analysis of a first-order upwind discretization of convection. Assuming the velocity is positive, then an upwind discretization of convection is,*

$$\frac{dU_j}{dt} + u \delta_x^- U_j = 0, \quad \Rightarrow \quad \frac{dU_j}{dt} + \frac{u}{\Delta x} (U_j - U_{j-1}) = 0.$$

Following the steps outlined above, we substitute the Fourier mode,

$$\frac{d\hat{U}_m}{dt} e^{ij\beta_m} + \frac{u}{\Delta x} [e^{ij\beta_m} - e^{i(j-1)\beta_m}] \hat{U}_m = 0.$$

Factoring out the term $e^{ik_m j \Delta x}$ gives,

$$\frac{d\hat{U}_m}{dt} + \frac{u}{\Delta x} (1 - e^{-i\beta_m}) \hat{U}_m = 0.$$

Thus,

$$\lambda_m \Delta t = -\frac{u \Delta t}{\Delta x} (1 - e^{-i\beta_m}) = -\frac{u \Delta t}{\Delta x} (1 - \cos \beta_m + i \sin \beta_m).$$

The upwind approximation gives both a real and imaginary part to the eigenvalue. In fact, the imaginary part is identical to the central difference part. The real part is negative, thus the

upwind approximation adds stability (in the sense that a negative real eigenvalue corresponds to amplitudes that decrease in time). A plot of the eigenvalues is shown in Figure 10.3 for $u\Delta t/\Delta x = 1$. The Matlab script for generating these eigenvalue plots is given below:

```
bm = linspace(-pi,pi,21);
CFL = 1;
lamdt = -CFL*(1 - exp(-i*bm));

plot(lamdt,'*');
xlabel('Real \lambda\Delta t');
ylabel('Imag \lambda\Delta t');
grid on;
axis('equal');
```

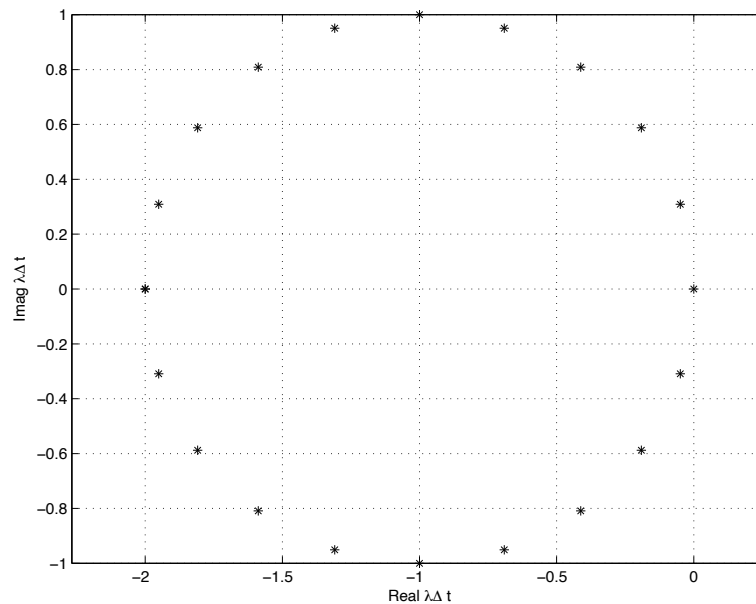


Figure 10.3: $\lambda_m \Delta t$ for a first-order upwind discretization of convection for $CFL = u\Delta t/\Delta x = 1$.